» neue fische × SPICED

Class Material

Week 4 - Data Processing Fundamentals

Day 2

Outline

1. Web Scraping Ethics & Regular Expressions

Section 1: Web Scraping & Regular Expressions

Learning Objectives

- Understand what web scraping is and when to use it vs APIs
- Use requests + BeautifulSoup to fetch, parse, and extract data
- Apply HTTP headers safely to reduce blocks/rate-limits
- Use regex to match/extract emails, phones, prices, and URLs
- Save data to CSV/JSON/DB and handle pagination & dynamic pages

Why Web Scraping?

- The web holds valuable data not always available via APIs
- Scraping = automated extraction of information from HTML pages
- Analogy:
  - **API** → asking the shopkeeper for a packed box of seashells
  - **Scraping** → going to the beach and collecting what you need
- Outcome: precise, flexible—but needs cleaning and care

Use Cases (Real-World)

- E-commerce: competitor prices, product catalogs, reviews
- Finance: news headlines, market tickers, filings
- Marketing: leads, emails (public), social proof
- Research: public records, scientific article metadata

The Scraping Workflow (5 Steps)

- **Request** → GET page HTML
- **Receive** → server returns HTML/CSS/JS
- **Parse** → turn HTML into a searchable tree
- **Extract** → pick elements/attributes you need
- **Store** → CSV/JSON/DB for analysis & pipelines

Core Tools

- **requests** – fetch pages.
- **BeautifulSoup (bs4)** – parse & navigate HTML.
- **re** – regex for pattern extraction/cleanup.
- **Selenium/Playwright** – render JS-heavy pages.
- **Scrapy** – production framework for large crawls.

Ethics, Legality & Politeness

- Check Terms of Service and robots.txt
- Respect rate limits (e.g., time.sleep, backoff)
- Avoid heavy crawling; throttle requests
- Never scrape private data or bypass authentication
- Identify yourself if possible; don’t overload servers

Headers Matter (Avoid Easy Blocks)

Why sites block bots: missing/odd headers

Common headers:

- User-Agent (must set)
- Accept, Accept-Language
- Referer
- Authorization / Cookies (for logged-in areas)

```
import requests
headers = {
  "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
  "Accept-Language": "en-US,en;q=0.9"
}
resp = requests.get("https://books.toscrape.com/", headers=headers)
print(resp.status_code, resp.text[:300])
```

BeautifulSoup Basics (Mental Model)

- Think of BeautifulSoup like a magnifying glass for HTML
- Build a tree, then select and extract

```
from bs4 import BeautifulSoup

html = """
<div class="product-card">
  <h2>Product Name</h2>
  <span class="price">$59.99</span>
</div>
"""
soup = BeautifulSoup(html, "html.parser")
name = soup.find("h2").get_text()
price = soup.find("span", class_="price").get_text()
print(name, price)
```

Selecting Elements (CSS-like)

- find, find_all, .text, .get('href')
- Classes/ids: class_="price", id="main"
- Chaining & scoping to reduce noise

```
cards = soup.find_all("div", class_="product-card")
for c in cards:
    title = c.find("h2").text.strip()
    price = c.find("span", class_="price").text.strip()
```

Hands-On: Quotes to Scrape

Target: `http://quotes.toscrape.com`

```python
import requests
from bs4 import BeautifulSoup

url = "http://quotes.toscrape.com"
r = requests.get(url)
soup = BeautifulSoup(r.text, "html.parser")

quotes = soup.find_all("span", class_="text")
authors = soup.find_all("small", class_="author")

data = [{"quote": q.text, "author": a.text} for q, a in zip(quotes, authors)]
for row in data[:3]:
    print(row)
```

Pagination Pattern

- Many lists span multiple pages (?page=2, next buttons)
- Identify “Next” link and loop until missing

```
import requests, time
from bs4 import BeautifulSoup

base = "http://quotes.toscrape.com"
url = base
all_quotes = []

while True:
    soup = BeautifulSoup(requests.get(url).text, "html.parser")
    for q in soup.select(".quote"):
        all_quotes.append({
          "text": q.select_one(".text").text,
          "author": q.select_one(".author").text
        })

    nxt = soup.select_one("li.next > a")
    if not nxt: break
    url = base + nxt.get("href")
    time.sleep(0.5)  # be polite

print("Collected:", len(all_quotes))
```

Storing Results (CSV/JSON/DB)

```python
import csv, json
# CSV
with open("quotes.csv","w",newline="",encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["text","author"])
    w.writeheader(); w.writerows(all_quotes)

# JSON
with open("quotes.json","w",encoding="utf-8") as f:
    json.dump(all_quotes, f, ensure_ascii=False, indent=2)
```

When Pages Are Dynamic

- If content loads via JS, requests may get only the shell
- Options:

➔ Inspect XHR calls (often a hidden JSON endpoint!)
➔ Use Selenium/Playwright to render, then scrape
➔ Prefer official APIs when available

Intro to Regex (Why + What)

- Regex = pattern language to find/extract/validate text
- Perfect for: emails, phones, prices, dates, IDs, URLs
- Think “magnet” that pulls only matching shapes from text

Regex Cheat Sheet (Mini)

Token	Meaning	Example
.	Any char (not newline)	c.t → cat/ cut
*	0+ of prev	a*b → b/ab/aab
+	1+ of prev	a+b → ab/aab
?	0/1 of prev	colou?r
[]	set/range	[A-Za-z]
\d	digit	\d+
\w	word char	\w+
^ / $	start / end	^Hello, world$
()	capture group	(\d{2})/(\d{2})/(\d{4})

Python re Essentials

```python
import re, textwrap
s = "Call +1-202-555-0173 or mail info@test.org, price: 19.99$"

print(re.findall(r"[\w.%+-]+@[\w.-]+\.\w+", s))  # emails
print(re.findall(r"\+?\d[\d\- ]{7,}\d", s))      # phones
m = re.search(r"(\d+\.\d{2})\$", s)              # price
print(m.group(1) if m else None)
```

- match (start), search (first), findall (list), finditer (iter), sub, split, compile

Scraping + Regex Together

```python
import requests, re
from bs4 import BeautifulSoup

u = "https://webscraper.io/test-sites"
html = requests.get(u).text
text = BeautifulSoup(html, "html.parser").get_text(" ")

emails = re.findall(r"[\w._%+-]+@[\w.-]+\.\w+", text)
phones = re.findall(r"\+?\d[\d\- ]{7,}\d", text)
print("Emails:", emails)
print("Phones:", phones)
```

- Use self-imposed limits; public pages only

Cleaning & Normalizing

- Strip whitespace, fix encodings, standardize price/number formats
- Deduplicate records (title + URL key)
- Validate with regex (e.g., email domain) before saving

Common Pitfalls & Fixes

- **403/429** → set headers; slow down; rotate IPs (ethically)
- **Empty content** → JS-rendered; inspect network tab / use Selenium
- **Wrong selectors** → site changed; re-inspect HTML
- **Encoding issues** → set encoding='utf-8', .text.strip()
- **Duplicate rows** → use canonical URL keys & sets

Choosing: API vs Scraping (Rule of Thumb)

Aspect	API	Scraping
Format	JSON/XML (Stable)	HTML(fragile)
Reliability	High	Can break on layout changes
Speed	Fast	Slower(parsing and rendering)
Limits	Documented rate limits	Anti-bot blocks possible
Legal	Clear ToS	Must check ToS & robots.txt

neue fische
SPICED